

FULL STACK DEVELOPMENT

FASE 5 | AULA 07 -

AZURE CONTAINER INSTANCE (ACI)

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
O QUE VOCÊ VIU NESTA AULA?	12
REFERÊNCIAS.....	13

EMSE

O QUE VEM POR AÍ?

Boas-vindas, alunos e alunas.

Nesta aula, vocês aprenderão sobre o Azure Container Instance (ACI), como usá-lo da forma correta, criar uma task para ser executada via Azure Pipelines e executar o deploy no Azure de forma automática.

Bons estudos!

EMSE

HANDS ON

Nesta aula, faremos a criação de um container por meio do Azure Pipelines, em forma de código na pipeline criada, e executaremos um deploy no Azure.



SAIBA MAIS

Neste documento de aula, faremos o deploy de nossa aplicação a partir do Azure Pipelines e do código hospedado no Azure Repos.

Dessa maneira, todo novo push no Repos iniciará a pipeline em que será criada a imagem docker e enviará para o Azure Container Registry de forma automática, para que não nos preocupemos com essa entrega. Em seguida, se iniciará nosso deploy no container criado. É necessário ter Azure Repos, Azure Pipelines e Azure Container Registry criados, além do Azure Container Instance para executar os passos a seguir.

Vamos partir da premissa de que o Repos, o Pipelines e o ACR já foram criados a partir de aulas feitas anteriormente. Então, agora precisamos editar a pipeline adicionando o stage que faz o deploy da aplicação em um container existente.

O primeiro passo é editar a pipeline existente e selecionar a opção “Azure CLI” na lateral direita da tela em “Tasks”. Em “Azure Resource Manager Connection”, selecione a subscription utilizada para o deploy no Azure. Em “Script Type”, selecione qual terminal você usará para executar o comando de deploy: “Powershell”.

Agora, em “Script Location”, selecione “Inline Script”; ele abrirá um campo para inserirmos o comando necessário para deploy. Então, copie o código a seguir. Podemos notar que temos diversas variáveis, isso ocorre para que você possa utilizar os valores relacionados à sua conta Azure e sua aplicação criada.

```
'az container create -g $(resourceGroup)
--name $(appName)
--image $(acrLoginServer)/$(acrName):$(Build.BuildId)
--cpu 1
--memory 1
--registry-login-server $(acrLoginServer)
--registry-username $(acrName)
--registry-password $(acrPassword)
--ports 80'
```

**** Todo script deve estar na mesma linha na pipeline; o código acima pula linhas para ficar legível.**

As variáveis devem ter os seguintes valores:

- **resourceGroup** = nome do resource group na Azure.
- **appName** = nome do container escolhido.
- **acrLoginServer** = endereço do repositório ACR criado; costuma terminar com azurecr.io.
- **acrName** = nome do repositório ACR: número da imagem baseada no número de build executado.
- **acrUsername** = usuário criado no ACR.
- **acrPassword** = senha criada no ACR.

A figura 1 traz um modelo de como as variáveis devem se parecer.

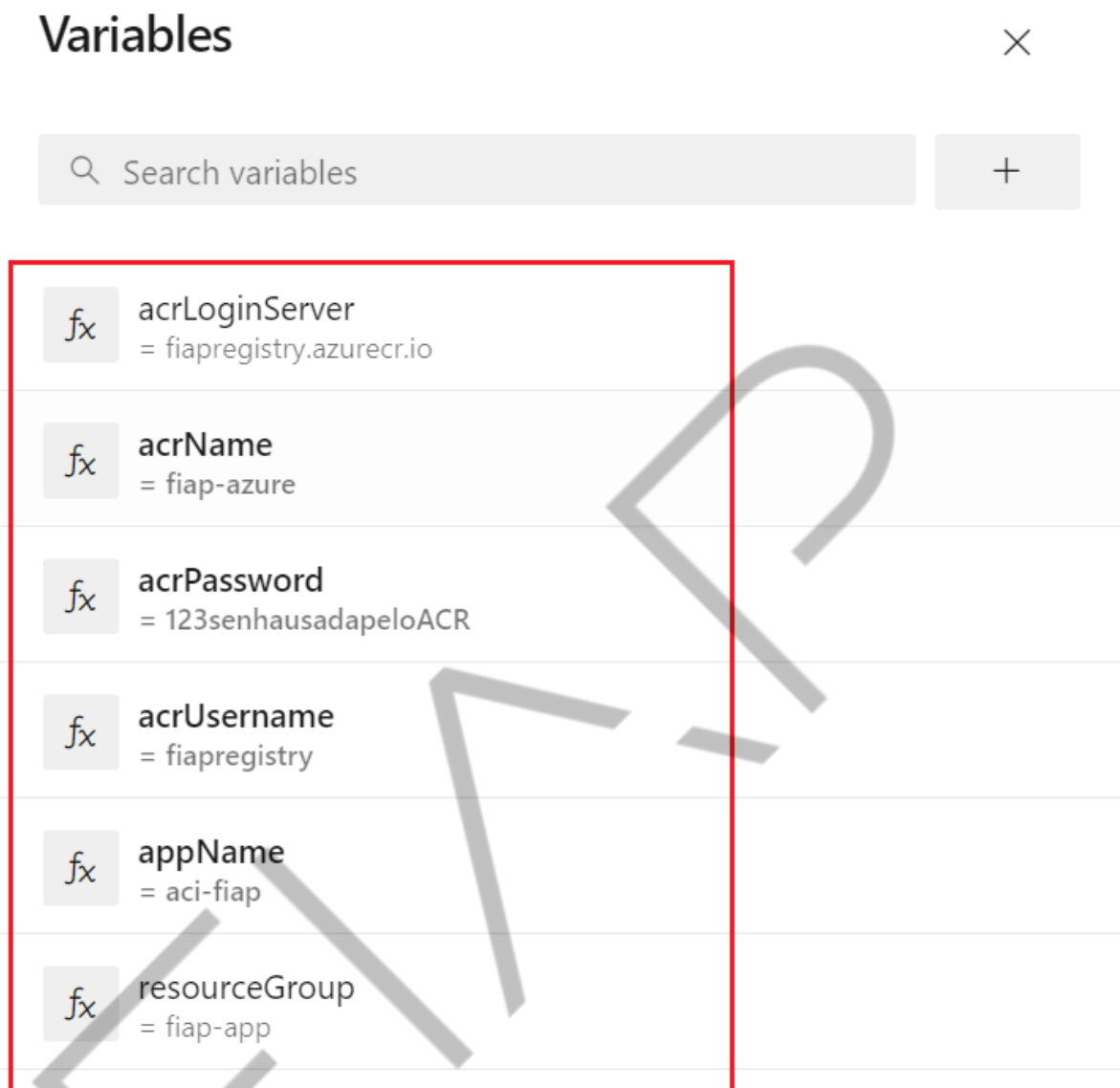


Figura 1 – Pipelines Variables
Fonte: Elaborado pelo autor (2024)

Para obter o valor das variáveis `acrUsername` e `acrPassword`, basta ir ao Azure, navegar até o serviço Azure Container Registry, entrar no repositório e em seguida “Chaves de Acesso”. Pegue os valores “Nome de Usuário” e “Password”, conforme a figura 2.

Página inicial > Registros de contêiner > fiapregistry

fiapregistry | Chaves de acesso

Pesquisar

Nome do Registro: fiapregistry

Servidor de logon: fiapregistry.azurecr.io

Usuário administrador: ☒

Nome de usuário: fiapregistry

Nome	Senha	Regenerar
password	[Redacted]	Regenerar
password2	[Redacted]	Regenerar

Figura 2 – Chaves de acesso
Fonte: Elaborado pelo autor (2024)

Depois de inserir todos os valores nas variáveis, basta salvar e voltar para o código da pipeline. A pipeline deve ficar parecida com a seguinte:

```
73
74 - stage: 'Dev'
75   displayName: 'Deploy to the dev environment'
76   dependsOn: Build
77   condition: succeeded()
78   jobs:
79     - deployment: Deploy
80       pool: self-hosted
81       environment: dev
82       variables:
83         - group: 'Release'
84       strategy:
85         runOnce:
86           deploy:
87             steps:
88               - download: current
89               - artifact: drop
90
91       Settings
92       task: AzureCLI@2
93       inputs:
94         azureSubscription: 'Azure subscription 1(2e3695b2-20e9-45c7-83cb-f561c6151b28)'
95         scriptType: 'ps'
96         scriptLocation: 'inlineScript'
97         inlineScript: 'az container create -g $(resourceGroup) \
98           --name $(appName) \
99           --image $(acrLoginServer)/$(acrName):$(Build.BuildId) \
100           --cpu 1 \
101           --memory 1 \
102           --registry-login-server $(acrLoginServer) \
103           --registry-username $(acrUsername) \
104           --registry-password $(acrPassword) \
105           --ports 80'
```

Figura 3 – ACI Pipelines
Fonte: Elaborado pelo autor (2024)

Neste exemplo, faremos o deploy no stage Dev e precisamos nos atentar a CPU, Memória e Portas, pois esses valores estão fora de variáveis (poderiam estar). Lembre-se de alterar tais valores se necessário.

Agora podemos salvar a pipeline e fazer com que ela seja executada. Vamos observar os passos e por fim checar no Azure se o container foi criado corretamente.

Nossa pipeline foi executada com sucesso, conforme a figura 4.

The screenshot displays the Azure DevOps pipeline interface. On the left, a sidebar shows the pipeline stages: 'Build the web application', 'Deploy to the dev environment', and 'Deploy to the staging environment'. The 'Deploy' stage is expanded, showing tasks: 'Initialize job', 'Download', 'AzureCLI' (highlighted with a red box), and 'Finalize Job'. The 'AzureCLI' task is currently selected, and its details are shown on the right. The task details include a title 'AzureCLI', a description, version '2.230.0', author 'Microsoft Corporation', and a help link. Below this, the task's output is displayed as a log, showing the execution of the 'az' command and the successful setup of the Azure CLI environment.

```
1 Starting: AzureCLI
2 =====
3 Task      : Azure CLI
4 Description : Run Azure CLI commands against an Azure subscription in a PowerShell Core/Shell script when running on Linux agent
5 Version    : 2.230.0
6 Author     : Microsoft Corporation
7 Help       : https://docs.microsoft.com/azure/devops/pipelines/tasks/deploy/azure-cli
8 =====
9 /usr/bin/az --version
10 azure-cli                2.56.0
11
12 core                      2.56.0
13 telemetry                 1.1.0
14
15 Dependencies:
16 msal                      1.24.0b2
17 azure-mgmt-resource       23.1.0b2
18
19 Python location '/opt/az/bin/python3'
20 Extensions directory '/home/fiap/.azure/cliextensions'
21
22 Python (Linux) 3.11.5 (main, Jan  8 2024, 09:08:13) [GCC 11.4.0]
23
24 Legal docs and information: aka.ms/AzureCliLegal
25
26
27 Your CLI is up-to-date.
28 Setting AZURE_CONFIG_DIR env variable to: /home/fiap/myagent/_work/_temp/.azclitask
29 Setting active cloud to: AzureCloud
30 /usr/bin/az cloud set -n AzureCloud
31 /usr/bin/az login --service-principal -u *** --password=*** --tenant 4f5d57b7-3db3-40cd-afca-b74d7f8788ac --allow-no-subscription
32 [
33 {
34   "cloudName": "AzureCloud",
35   "homeYamlPath": "4f5d57b7-3db3-40cd-afca-b74d7f8788ac"
```

Figura 4 – Stage Deploy
Fonte: Elaborado pelo autor (2024)

Vamos na Azure checar se nossa aplicação foi criada utilizando as informações corretas. Na página inicial, vamos pesquisar por “Grupos de Recursos” e selecionar essa opção.

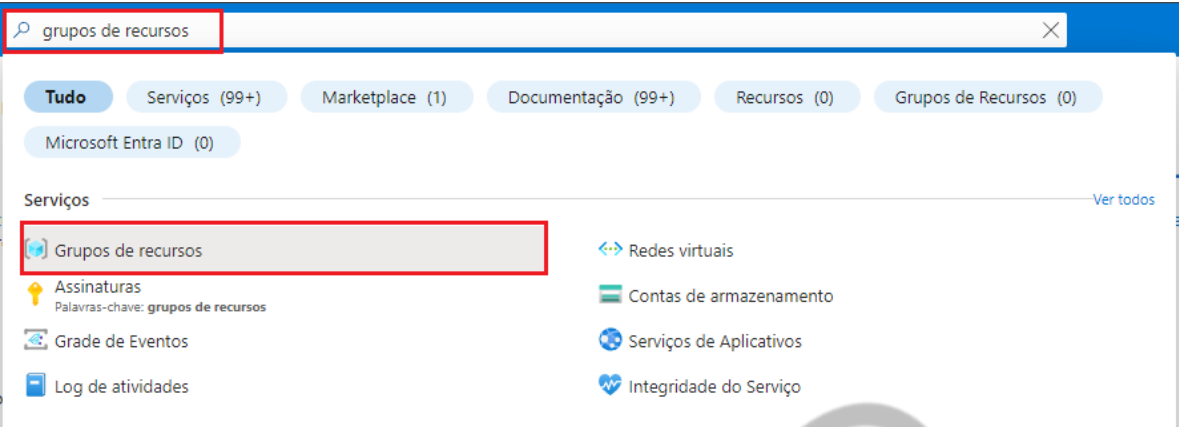


Figura 5 – Grupos de recursos
Fonte: Elaborado pelo autor (2024)

Selecione o grupo de recursos em que a aplicação foi criada e procure-a na lista. Baseada em nossas variáveis, a aplicação criada foi a aci-fiap.

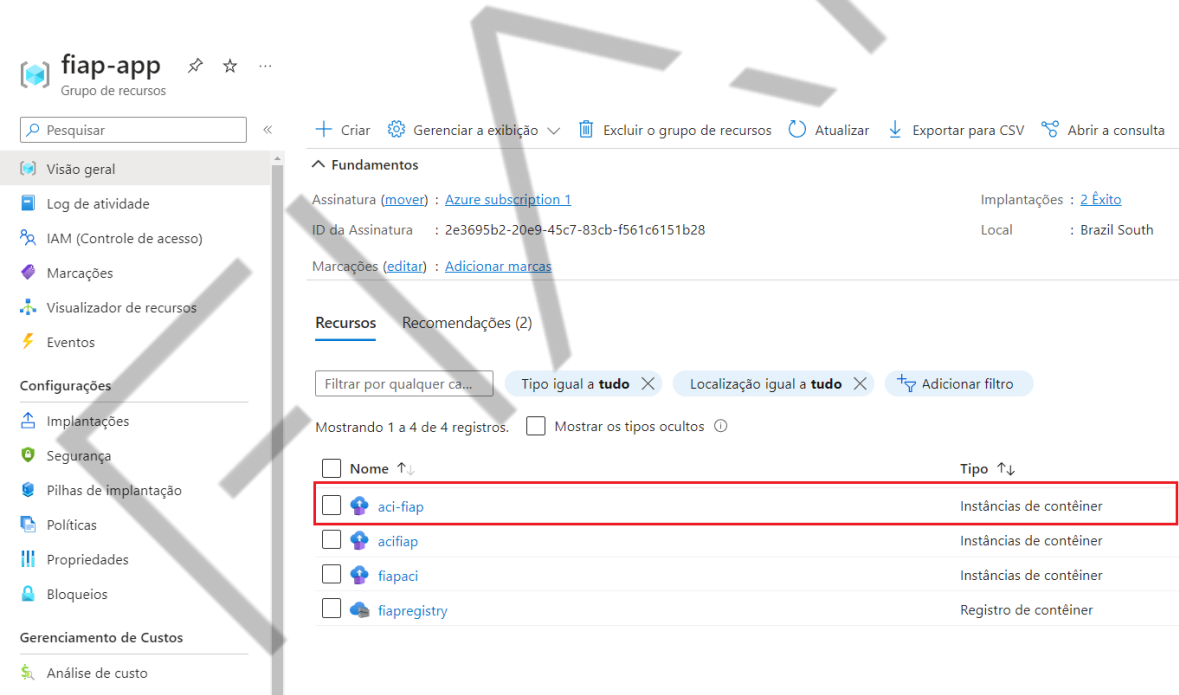


Figura 6 – aci-fiap
Fonte: Elaborado pelo autor (2024)

Podemos checar as informações entrando em container e propriedades.

aci-fiap | Contêineres

Instâncias de contêiner

Pesquisar

Atualizar

Enviar comentários

Visão geral

Log de atividade

IAM (Controle de acesso)

Marcações

Configurações

Contêineres

Identidade

Propriedades

Bloqueios

Monitoramento

Métrica

Alertas

Diagnostic Settings (Preview)

Automação

CLI / PS

Tarefas (versão prévia)

Exportar modelo

Um contêiner e 0 contêineres de inicialização

Nome	Imagem	Estado
aci-fiap	fiapregistry.azurecr.io/fiap-azure:58	Waiting

Eventos

Propriedades

Logs

Conectar

Nome

aci-fiap

Imagem

fiapregistry.azurecr.io/fiap-azure:58

Portas

Núcleos da CPU

1

Memória

1 GiB

SKU da GPU

None

Contagem de GPUs

Figura 7 – Container
Fonte: Elaborado pelo autor (2024)

Chegamos ao fim de mais uma aula, espero que vocês tenham gostado do conteúdo e aprendido com ele.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, você viu como fazer um deploy no Azure Container Instance a partir do Azure Pipelines de forma automatizada, atualizando a aplicação a cada nova imagem criada por meio de alterações nos códigos feitos pelo Azure Repos.



REFERÊNCIAS

MICROSOFT BUILD. **Build and push Docker images to Azure Container Registry using Docker templates.** 30/01/2023. Disponível em: <<https://learn.microsoft.com/en-us/azure/devops/pipelines/ecosystems/containers/acr-template?view=azure-devops>>. Acesso em: 29 abr. 2024.

MICROSOFT BUILD. **Início Rápido:** Implantar uma instância de contêiner no Azure usando o portal do Azure. 05/01/2024. Disponível em: <<https://learn.microsoft.com/pt-br/azure/container-instances/container-instances-quickstart-portal>>. Acesso em: 29 abr. 2024.

MICROSOFT BUILD. **Limites de cota e disponibilidade de recursos para ACI.** 2024. Disponível em: <<https://learn.microsoft.com/pt-br/azure/container-instances/container-instances-region-availability>>. Acesso em: 29 abr. 2024.

PALAVRAS-CHAVE

Palavras-chave: Docker. ACR. Azure Pipelines. ACI.

EMSE

POS TECH